Lecture 01-b
DFA-minimization

1st 2019-2020

Lexical Analysis:
DFA Minimization

Minimizing DFA's

By Partitioning

Minimizing DFA's

- ***Lots*** of methods
- All involve finding **equivalent states**:
  - States that go to equivalent states under all inputs (sounds recursive)
- We will use the ***Partitioning Method***

DFA → Minimal DFA

- Hopcroft's algorithm

Remember DFA =(Q,Σ,δ,q_0,F)

Initial partition, P_0, has two sets: *$\{D_F\}$ and $\{D\text{-}D_F\}$*

Splitting a set *s* ("partitioning a set by $\underline{a}$")

- Assume q_i, & $q_j \in s$ and $\delta(q_i,\underline{a}) = q_x$ and $\delta(q_j,\underline{a}) = q_y$
- If q_x and q_y are not in the same set, then *s* must be split
  - → q_i has transition on a, q_j does not ⇒ $\underline{a}$ splits *s*
- One state in the final DFA cannot have two transitions on $\underline{a}$ (otherwise we have an NFA!)

$P \leftarrow \{ D_F, \{D-D_F\}\}$

while (P is still changing)

$T \leftarrow \emptyset$

for each set $p \in P$

$T \leftarrow T \cup Split(p)$

$P \leftarrow T$

Split(S)

for each $\alpha \in \Sigma$

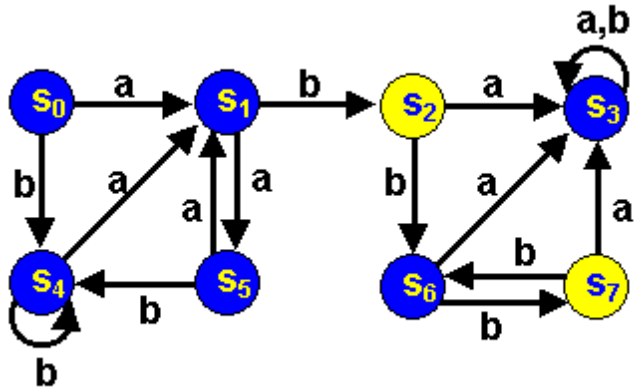
if α *splits S into* s_1 *and* s_2

then return $\{s_1, s_2\}$

return S

Example-
Minimizing DFA's by Partitioning

Consider the following dfa :



- **Accepting** states are **{S2, S7}**
- Non-accepting states are **{S0, S1, S3, S4, S5, S6}**
- Are any states really the same?

Partitioning Algorithm

- First
  - Divide the set of states into

    Final and
    Non-final states

Partition I

Partition II

	a	*b*
S_0	S_1	S_4
S_1	S_5	S_2
S_3	S_3	S_3
S_4	S_1	S_4
S_5	S_1	S_4
S_6	S_3	S_7
$*S_2$	S_3	S_6
$*S_7$	S_3	S_6

Partitioning Algorithm

- Now
  - See if states in each partition each go to the same partition
- S_1 & S_6 are different from the rest of the states in Partition I (but like each other)
- We will move them to their own partition

	a	***b***
$\mathbf{S_0}$	S_1 **I**	S_4 **I**
$\boldsymbol{S_1}$	S_5 ***I***	S_2 ***II***
$\mathbf{S_3}$	S_3 **I**	S_3 **I**
$\mathbf{S_4}$	S_1 **I**	S_4 **I**
$\mathbf{S_5}$	S_1 **I**	S_4 **I**
$\boldsymbol{S_6}$	S_3 ***I***	S_7 ***II***
$\mathbf{*S_2}$	$\mathbf{S_3}$ **I**	$\mathbf{S_6}$ **I**
$\mathbf{*S_7}$	$\mathbf{S_3}$ **I**	$\mathbf{S_6}$ **I**

Partitioning Algorithm

- Now again
  - See if states in each partition each go to the same partition
  - In Partition I, S_3 goes to a different partition from S_0, S_5 and S_4
  - **We'll move S3 to its own partition**

	a	*b*
S_0	S_1 III	S_4 I
S_5	S_1 III	S_4 I
S_3	S_3 I	S_3 I
S_4	S_1 III	S_4 I
S_1	S_5 I	S_2 II
S_6	S_3 I	S_7 II
*S_2	S_3 I	S_6 III
*S_7	S_3 I	S_6 III

Partitioning Algorithm

Note changes in S_6, S_2 and S_7

	a	*b*
S_0	S_1 III	S_4 I
S_5	S_1 III	S_4 I
S_4	S_1 III	S_4 I
S_3	S_3 IV	S_3 IV
S_1	S_5 I	S_2 II
S_6	S_3 IV	S_7 II
*S_2	S_3 IV	S_6 III
*S_7	S_3 IV	S_6 III

Partitioning Algorithm

- **Now S_6 goes to a different partition on an *a* from S_1**
- **S_6 gets its own partition.**
- **We now have 5 partitions**
- **Note changes in S_2 and S_7**

	a	*b*
S_0	S_1 III	S_4 I
S_5	S_1 III	S_4 I
S_4	S_1 III	S_4 I
S_3	S_3 IV	S_3 IV
S_1	S_5 I	S_2 II
S_6	S_3 IV	S_7 II
$*S_2$	S_3 IV	S_6 V
$*S_7$	S_3 IV	S_6 V

Partitioning Algorithm

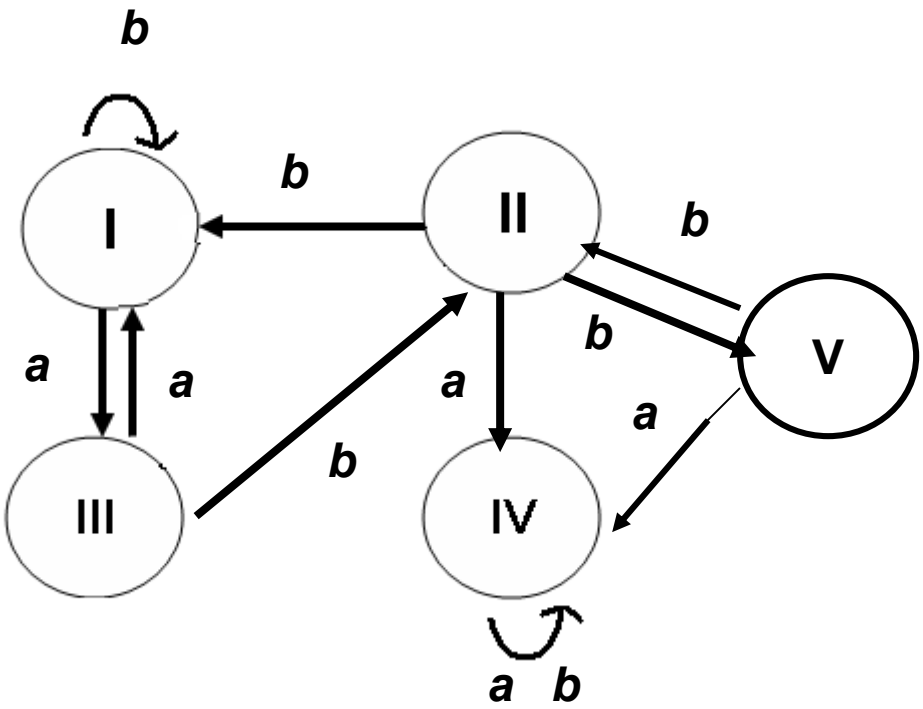
- All states within each of the 5 partitions are identical.
- We might as well call the states I, II III, IV and V.

	a	*b*
S_0	S_1 III	S_4 I
S_5	S_1 III	S_4 I
S_4	S_1 III	S_4 I
S_3	S_3 IV	S_3 IV
S_1	S_5 I	S_2 II
S_6	S_3 IV	S_7 II
$*S_2$	S_3 IV	S_6 V
$*S_7$	S_3 IV	S_6 V

Partitioning Algorithm

Here they are:

	a	*b*
I	III	I
*II	IV	V
III	I	II
IV	IV	IV
V	IV	II


b
I
b
II
b
V
b
a
a
a
a
b
III
IV
a
b